University of Cape Town ECO5040W - Financial Software Engineering - Class Project Brief - XRPL-Based FX Remittance Platform Using RLUSD

Release Date: 2026-08-14
Due Date: 2026-09-25

1. Background

Stablecoins are crypto assets designed to maintain a relatively stable value by being linked to a reference asset, most commonly a fiat currency such as the US dollar (USD) or the South African Rand (ZAR). In cross-border remittances, stablecoins can serve as a settlement instrument that allows value to move between countries at high speed, in real time, and at lower cost than many traditional correspondent-banking channels. They can also reduce reliance on multiple intermediaries, improve transaction traceability, and enable recipients to access funds through digital wallets before converting them into local currency or cash. However, their use in remittances still depends on reliable fiat cash-in and cash-out networks, sufficient liquidity, secure custody arrangements, and compliance with anti-money-laundering, foreign-exchange, consumer-protection, and payment regulations.

The objective of this assignment is to design, implement, test, and demonstrate a prototype cross-border foreign-exchange remittance platform that operates in a manner broadly comparable to services such as MoneyGram or Western Union.

2. Project Overview

Students are required to design and develop a prototype foreign-exchange remittance platform similar to MoneyGram or Western Union. The platform must use **RLUSD stablecoin on the XRP Ledger Testnet** as the digital settlement asset.

The system should allow a sender in South Africa to initiate a remittance using South African rand. The recipient should receive the equivalent value in RLUSD through a custodial web wallet. The recipient may then choose to retain the RLUSD or request a simulated cash-out in US dollars or another supported fiat currency.

The platform is an academic prototype only. No real customer funds, real remittances, or production blockchain credentials may be used. The FX application is expected to be implemented using a modular architecture style, utilising a Python Framework like

Flask, Django or FastAPI; and a relational SQL database (e.g. SQLite, Postgres, MySQL).

3. Core User Journey

The FX platform should support the following illustrative remittance journey:

1. A sender registers and logs in.
2. The sender completes a mock Know Your Customer process.
3. The sender creates or selects a recipient.
4. The sender enters the ZAR amount to remit.
5. The platform retrieves or simulates the current USD/ZAR exchange rate.
6. The system calculates:
    - transaction fees;
    - the exchange-rate margin;
    - the net amount being converted; and
    - the RLUSD amount that the recipient will receive.
7. The sender confirms a simulated ZAR cash-in payment.
8. A settlement request is placed on a message queue.
9. A background worker transfers RLUSD on the XRP Ledger Testnet.
10. The recipient logs in to the web platform and views the received RLUSD.
11. The recipient requests a simulated cash-out in USD or another supported fiat currency.

4. Functional Scope

The project must include the following core features.

User Registration and Login

The platform must allow users to:

- register;
- log in and log out;
- manage basic profile information;
- view their KYC status;
- view their transaction limits; and
- view their wallet balance and transaction history.

Passwords must be securely hashed and must never be stored in plaintext.

Mock KYC

The platform must implement a simplified KYC process that collects:

- full name;
- date of birth;
- nationality;
- identification number;
- residential address;
- mobile number;
- email address; and
- source of funds.

An administrator must be able to approve or reject the KYC application.

Only approved users may send remittances.

Beneficiary Management

A sender must be able to add and view beneficiaries.

A beneficiary record should contain:

- full name;
- mobile number or email address;
- country;
- preferred payout currency; and
- relationship to the sender.

Remittance Limits

The FX system must enforce configurable daily and monthly remittance limits.

For example:

User level	Daily limit	Monthly limit
Unverified	ZAR 0	ZAR 0
Verified	ZAR 3,000	ZAR 25,000

The system must reject a remittance that would cause the user to exceed either limit.

Exchange Rates and Fees

The FX platform must use the USD/ZAR exchange rate applicable when the transaction is created.

Students may use:

- a public exchange-rate API;
- a mock exchange-rate service; or
- a manually configured exchange-rate table.

The transaction quotation must show:

- ZAR send amount;
- exchange rate;
- transaction fee;
- foreign-exchange margin;
- RLUSD amount to be received;
- cash-out fee; and
- estimated recipient payout.

A simple fee model may include:

- a fixed remittance fee;
- a percentage-based fee;
- a foreign-exchange margin; and
- a cash-out fee.

All fees must be configurable.

Simulated ZAR Cash-In

The FX platform must simulate the sender paying ZAR through one method, such as:

- cash at an agent;
- bank transfer; or
- card payment.

An administrator or mock payment service may confirm that the payment has been received.

The RLUSD transfer must not begin until the ZAR cash-in has been confirmed.

Custodial RLUSD Wallet

The recipient must have access to a custodial web wallet.

The wallet should display:

- available RLUSD balance;
- incoming transfers;
- outgoing or cash-out transactions;
- transaction status;
- transaction date; and
- XRP Ledger transaction hash.

Students may use either:

- a separate XRPL Testnet account for each user; or
- one platform wallet with customer balances maintained in an internal database ledger.
- RLUSD is an issued token on XRPL, therefore wallets need a `TrustSet` to the RLUSD issuer in order to hold it. (Explore TrustSet and understand TrustLines to work with RLUSD)

The selected approach must be explained in the technical specification.

XRPL Testnet Integration

The FX platform must integrate with the XRP Ledger Testnet using an appropriate software development kit.

The implementation must demonstrate:

- XRPL Testnet account setup;
- RLUSD or lecturer-approved test-token transfer;
- transaction signing;
- transaction submission;
- transaction-hash storage;
- successful transaction validation; and
- failed-transaction handling.

No Mainnet accounts or real RLUSD must be used.

Message Queue

RLUSD transfers must be processed asynchronously through a message queue such as RabbitMQ, Kafka, Redis Streams, or a similar message queue.

The simplified process should be:

1. ZAR payment is confirmed.
2. A settlement message is added to the queue.
3. A worker reads the message.
4. The worker submits the RLUSD transfer to XRPL Testnet.
5. The system records whether the transaction succeeded or failed.
6. The recipient's wallet balance is updated.

The FX system must prevent duplicate messages from crediting the recipient more than once.

Private-Key Security

Any XRPL private keys stored in the database must be encrypted.

The encryption key must not be stored in the same database as the encrypted private keys.

Private keys must:

- never be returned through the API;
- never appear in logs;
- never be committed to source control; and
- only be decrypted by the component responsible for signing XRPL transactions.

Simulated Cash-Out

The recipient must be able to request a cash-out from RLUSD to:

- USD; or
- A supported local fiat currency.

The FX platform must calculate the fiat amount using the applicable exchange rate and deduct the configured cash-out fee.

The cash-out itself may be simulated using a status such as:

- requested;

- approved;
- completed; or
- failed.

5. Technical Architecture

The proposed FX platform should contain the following main components:

- web front end;
- REST API;
- relational database;
- user and KYC module;
- remittance and fee module;
- wallet and transaction module;
- exchange-rate service;
- message broker;
- XRPL settlement worker;
- mock cash-in and cash-out services; and
- administrator interface.

The API must use JSON and should optionally provide interactive OpenAPI or Swagger documentation.

Additional Considerations and **Useful Resources**

- To get free XRP on the XRP Ledger TestNet, make use of faucets. (you can get some free XRP from a TestNet faucet https://xrpl.org/resources/dev-tools/xrp-faucets).
- You can deploy the Python backend(s) for your system to cloud providers such as Render[1], Railway[2] etc.
- Generate synthetic users for your performance tests and simulations.
- Additional technical resources:
    - https://help.xaman.app/app/learning-more-about-xaman/how-to-access-testnet-on-xrp-ledger
    - https://xrpl.org/resources/dev-tools/xrp-faucets
    - https://xrpl.org/docs/tutorials/python/build-apps/get-started
    - https://xrpl.services/

6. Regulatory and Business Considerations

The project documentation must briefly discuss:

[1] https://render.com/
[2] https://railway.com/

- KYC and anti-money-laundering requirements;
- transaction monitoring;
- customer transaction limits;
- protection of customer information;
- custody of crypto assets;
- stablecoin and crypto-asset regulation;
- foreign-exchange and capital-flow controls;
- consumer protection;
- safeguarding of customer funds; and
- licensing that may be required for a real remittance service.

Students are not required to provide a full legal opinion. They must demonstrate an understanding that using a stablecoin does not remove the need to comply with financial-services, payment, remittance, and exchange-control rules.

7. Project Deliverables

Each team must submit the following:

i. Business and Technical Specification

A combined document of approximately 10 to 15 pages containing:

- business problem;
- user journey;
- functional requirements;
- fee model;
- exchange-rate calculation;
- remittance limits;
- architecture diagram;
- cash-in flow;
- RLUSD settlement flow;
- cash-out flow;
- database design;
- API overview;
- security design;
- regulatory considerations; and
- assumptions and limitations.

ii. Working Web Application

The application must demonstrate:

- registration and login;
- mock KYC;
- beneficiary creation;
- exchange-rate quotation;
- fee calculation;
- daily and monthly limits;
- simulated ZAR cash-in;
- queued RLUSD transfer;
- recipient wallet;
- simulated cash-out;
- administrator approval functions; and
- encrypted XRPL private keys.

iii. Presentation

A presentation of approximately 10 to 15 slides covering:

- business problem;
- proposed solution;
- user journey;
- architecture;
- fee model;
- system demonstration;
- security;
- regulatory considerations;
- testing results; and
- lessons learned.

iv. Performance Testing Results

Students must test and report:

- API response times;
- number of requests processed per second;
- message-queue throughput;
- RLUSD transaction processing time;
- transaction success and failure rates; and
- system behaviour under concurrent use.

The report should include a small number of tables or charts and explain any performance bottlenecks identified.

The project will culminate in a presentation and a live demo by each team.

8. Project Administration

The final FX solution should demonstrate how a modern remittance platform can accept a simulated ZAR payment, settle value using RLUSD on the XRP Ledger, provide the recipient with access to a custodial stablecoin wallet, and support a simulated conversion back into fiat currency.

Outcomes

The purpose of the project is to expose students to the technical, business, financial, security, and regulatory considerations involved in building a modern cross-border remittance platform.

On completion of the project, students should be able to demonstrate competence in:

- full-stack FinTech software development;
- foreign-exchange calculations;
- blockchain integration;
- custodial wallet architecture;
- asynchronous transaction processing;
- API development;
- database security;
- identity and access management;
- transaction monitoring;
- regulatory technology;
- software testing;
- performance benchmarking; and
- technical and business communication.

Assessment

Component	Weight
Business and technical specification	20%
Web application functionality	35%
XRPL and message-queue integration	20%
Security and private-key protection	10%
Performance testing	10%

Presentation and demonstration	5%
Total	**100%**

Project Teams

First Name	Last Name	Student Number	Email	Group
Francesca	Behr	BHRFRA001	BHRFRA001@myuct.ac.za	1
Claire	Campbell	CMPCLA004	CMPCLA004@myuct.ac.za	1
Mridula	Kumar	KMRMRI001	KMRMRI001@myuct.ac.za	1
Raibim	Alam	ALMRAI001	ALMRAI001@myuct.ac.za	1
Sian	Caine	CNXSIA001	CNXSIA001@myuct.ac.za	2
Maesela	Sekoele	SKLMAE001	SKLMAE001@myuct.ac.za	2
Joseph	Valkin	VLKJOS001	VLKJOS001@myuct.ac.za	2
Realeboha	Motlamelle	MTLREA003	MTLREA003@myuct.ac.za	2
Annita	Ngoma	NGMANN002	NGMANN002@myuct.ac.za	3
Karabo	Tigedi	TGDMOE001	TGDMOE001@myuct.ac.za	3
Kerry-Lynn	Whyte	WHYKER001	WHYKER001@myuct.ac.za	3
Ndumiso	Zondi	ZNDNDU007	ZNDNDU007@myuct.ac.za	4
Marco	Klopper	KLPMAR012	KLPMAR012@myuct.ac.za	4
Muki	Mdluli	MDLMUK001	MDLMUK001@myuct.ac.za	4
Rafaela	Stevenson	STVRAF001	STVRAF001@myuct.ac.za	5
Lilitha	Mzamo	MZMLIL002	MZMLIL002@myuct.ac.za	5
Nikola	Milosavljevic	MLSNIK001	MLSNIK001@myuct.ac.za	5

Dates

Project dates below:

- Friday 14th August - Project brief released & team allocations
- Tuesday 18th August - Project Kick-off meeting
- Friday 21st August - Project Check-in
- Saturday 5th September - Mid-term vacation starts
- Sunday 13th September - Mid-term vacation ends

- Friday 18th September - Project Check-in (Technical specification complete, scaffolding for proof of concept started)
- Friday 25th September - Project complete and final presentation (Proof of concept demo, lessons learnt & final presentation complete)
- Project presentation date TBC

Questions can be sent to Julian Kanjere (University of Cape Town) - julian.kanjere@uct.ac.za and cc Marc Levin (University of Cape Town) - LVNMAR013@myuct.ac.za